Network Function Migration in Softwarization Based Networks with Mobile Edge Computing

Yijing Liu*, Gang Feng*, *Senior Member, IEEE*, Zhuo Chen†, Shuang Qin*, Guanqun Zhao*
*National Key Laboratory of Science and Technology on Communications,
University of Electronic Science and Technology of China
†College of Computer Science and Engineering, Chongqing University of Technology
E-mail:fenggang@uestc.edu.cn

***Abstract*—Network Function Virtualization has been widely acknowledged as one of the fundamental technologies for 5G and beyond by consolidating network functions into general-purpose hardware. To support a specific type of service, a virtualized network topology, such as Service Function Chain, is constructed by logically connecting a set of virtual network functions (VNFs). Meanwhile, Mobile Edge Computing (MEC) provides cloud resources at the edge of networks, meeting the stringent service requirements of many emerging mobile applications. With the widespread of new compute-intensive and Internet of Things applications, the amount of service flows in edge networks is rapidly increasing, causing network congestion easily because of the sinking of the computing capabilities. Moreover, due to the limited coverage of edge servers and erratic user mobility, it is difficult to maintain satisfactory service performance. Therefore, in order to support diverse services with various Quality of Service requirements, an online dynamic VNF migration is imperative in mobile networks. In this paper, we investigate the NF migration in softwarization based mobile networks with MEC, with the aim to minimize the number of link congestion in the networks. We first formulate the link congestion problem as a multidimensional knapsack problem, which is proved NP-hard. Then we resort to deep reinforcement learning to solve the online migration problem. Numerical results show that the proposed migration strategy can significantly reduce the number of link congestion and end-to-end service delay in comparison with the state-of-the-art solutions.**

I. Introduction

It is widely acknowledged that Software Define Network (SDN) and Network Function Virtualization (NFV) have been the key architectural technologies for 5G and beyond systems[1]. For significant cost reduction and ease of management, NFV decouples Network Functions (NFs) from proprietary hardware and deploys them as virtualized entities on general-purpose platforms, promising to reduce the network operator's capital expenditure and operational expenditure. Such virtual NFs (VNFs) interconnected by virtual links, commonly constitute Service Function Chains (SFCs) which predefine the order of network functions to process service flows. A key challenge of processing service flows is the limited capacity of commodity servers, which may increase performance overhead for software packet processing [1].

Three service categories in 5G have been identified by the Third Generation Partnership Project, including enhanced mobile broadband (eMBB), massive machine-type communications (mMTC), and ultra-reliable and low-latency communications (URLLC) [1]. Mobile services with different service categories request flexible allocations of network resources in response to network dynamics. Mobile Edge Computing (MEC) technique provides users with cloud computing capabilities and IT services in the Radio Access Network (RAN) (*i.e.*, the edge of mobile networks), meeting the stringent service delay requirements. To further shorten the routing paths of flows, MEC network can be divided into multiple clusters. However, with the rapid increase of the heterogeneous traffic from geographically distributed users, on the one hand, processing the services at MEC clusters with limited resources can be extremely stressful and thus increase the processing delay and link congestion. On the other hand, processing the services in remote clouds cannot meet the delay requirements for some delay-sensitive services. Therefore, intuitively we can exploit both edge servers and remote clouds in a collaborative way to support the service with different quality of service (QoS) requirements.

There is little known research work focusing on joint task processing at core and MEC in softwarization based mobile networks. In [2], the authors propose an adaptive user-managed service placement mechanism to run services on local devices, edge servers, and remote clouds. However, they consider only the shortest-paths without network congestion. The authors of [3] propose an edge federation model to integrate edge computing as well as remote clouds, focusing on the deployment problem. However, to the best authors' knowledge, there is no research work that considers the NF migration between the core clouds and MEC clouds. Indeed, a set of NFs of an SFC for processing a flow can be assigned multiple servers, at either core clouds or MEC. This implies that a flow service can be accomplished in a cooperative way by both the core clouds and MEC servers.

In this paper, we propose an intelligent VNF migration strategy for VNF based mobile networks. Both horizontal and vertical migrations are considered. Specifically, in the

This work has been supported by the General Program of National Natural Science Foundation of China (Grant No. 61871099), the Key Projects of National Natural Science Foundation of China (Grant No. 61631004), and the Fundamental Research Funds for the Central Universities (Grant No. ZYGX2019J122).

978-1-7281-5089-5/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.

horizontal direction, a VNF can be migrated from an MEC cluster to another for avoiding the overload at the local MEC cluster, or from a core cloud to another for reducing the transmission delay and propagation delay. In the vertical direction, a VNF can be migrated from an MEC cluster to a core cloud for alleviating the traffic pressure at the edge network, or verse versa for improving the experience of mobile services.

Due to the dynamic nature of the network state and external migration variables which are difficult to accurately characterize, there is no one-for-all solution for the long-term migration decision problems. Meanwhile, the future traffic should be taken into account to reduce the number of NF migration which definitely incur a certain cost. Fortunately, reinforcement learning (RL) can be exploited to solve such sequential decision problems under complex environments, with the aim to maximize the long-term reward by continuously interacting with the environment [4]. Although RL works well in decision-making scenarios when the size of the state-action space in the system is relatively small, the effectiveness of RL diminishes as the size of the state-action space becomes large. Then, deep reinforcement learning (DRL) emerges as a good alternative to solve the decision-making problem in the wireless system with a large size of service flows [4]. In this paper, we employ Double-Deep Q-Network (DDQN) to solve the migration problem, by introducing the experience relay pool and the target network to improve the stability of the training results. Numerical results show that our proposed migration strategy significantly outperforms the offline optimal service placement (O2SP) [2] and the greedy heuristic algorithm (GHA) [5] in link congestion performance.

The rest of the paper is organized as follows. Section II describes the system model. Section III formulates the VNF migration problem. In Section IV, we present the DDQN-based algorithm. Section V presents numerical results as well as discussion. Finally, Section VI concludes this paper.

II. System Model

A. Network Model

We consider a scenario in which the network with MEC is built upon an SDN/NFV-enabled 5G network infrastructure, as shown in Fig. 1. Similar to that in NORMA [6], let there be two SDN-based controllers which are used to translate decisions of the control applications into commands to VNFs. One controller is used to manage the local NFs while the other one is used to manage the cooperation between MEC clusters and remote clouds. Moreover, in the edge network, each MEC cluster has a Local Controller (LC) node that manages other member nodes and is also responsible for the flow forwarding in the cluster.

The network topology is modeled as an edge- and vertex-weighted, undirected graph $\mathcal{G} =< \mathcal{V}, \mathcal{L} >$, where $\mathcal{V}$ and $\mathcal{L}$ represent the set of substrate nodes and links respectively. Let $g = 1$ indicate the edge network and $g = 2$ the core network. We assume that a specific node is denoted by $V^k_{g,u}$, where $V^k_{1,u}$ represents the substrate node k in the uth cluster When $g = 1$

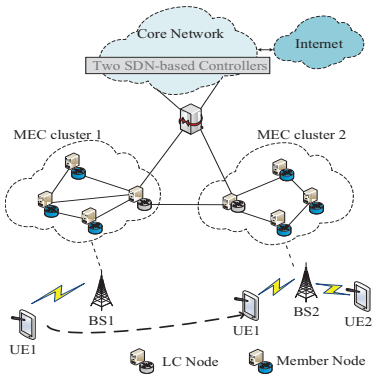

Fig. 1. The softwarization based network model.

and $V^k_{2,u}$ represents the substrate node k which is placed at uth core cloud when $g = 2$. Let the number of MEC clusters be y at the edge network. Meanwhile, we assume that the substrate nodes in the core network are sufficient enough. Let $B_{k,w}$ and $L_{k,w}$ represent the currently available link bandwidth and the substrate link distance between node $V^k_{g,u}$ and $V^w_{g,u}$ respectively. Moreover, the computation resource of the node $V^k_{g,u}$ is denoted by $C^k_{g,u}$.

In the softwarization based network, a data flow needs a specific service which is defined by the SFC. Let there be h service requests arriving within time T, and denote $\mathcal{H} = \{d_1, d_2, ..., d_h\}$. For a specific request d_i, we assume that the maximum tolerated delay is denoted by ${t_i}^{\max}$. In addition, the import and export network elements of d_i are represented by I_i and E_i respectively. P_i denotes the path from I_i to E_i where the flow d_i will traverse. Service request $d_i \in \mathcal{H}$ to be fulfilled by an SFC can be represented by $S_i = \{S_{i,1}, S_{i,2}, \ldots, S_{i,j}, \ldots, S_{i,K}\}, 1 \leq j \leq K$, where $S_{i,j}$ can be an instance of either a single VNF or parallel VNFs.

The set of VNF types in the serving SFCs within time T is denoted by $\mathcal{F}$. We use x to represent the specific VNF, where $x \in \mathcal{F}$. The computing resource of VNF x allocated to flow d_i is denoted by ω^i_x. Let $\beta^i_{x,x'}$ be the virtual link bandwidth between x and x' in S_i.

B. Link Congestion

Compared with processing multi-flows at MEC clusters only, NF migration between core clouds and MEC clusters can balance the processing efficiency of delay-sensitive services and improve the overall efficiency of the system, which requires effective cooperation between MEC clusters and core clouds. The overload of processing tasks at MEC clusters is likely to cause congestion and generate additional latency. Thus we can set our design objective as minimizing the number of link congestion, so as to maximize resource efficiency subject to service quality requirement in softwarization based mobile networks with MEC.

We first define the binary migration decision variable $\varepsilon^{g,u,k}_{x,t} \in \{0, 1\}$ (*i.e.*, $\varepsilon^{g,u,k}_{x,t} = 1$ indicates that VNF x is migrated to the substrate node $V^k_{g,u}$ at tth time slot, and no migration otherwise). Additionally, we introduce the binary decision variable $\sigma^{t,k,w}_{i,x,x'} \in \{0, 1\}$ to indicate whether the virtual link $\beta^i_{x,x'}$ is mapped onto substrate link (k, w) at tth time slot or not: $\sigma^{t,k,w}_{i,x,x'} = 1$ yes and 0 otherwise.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.

We set a bandwidth threshold for each link to indicate the threshold for determining link congestion. In this paper, we use the matrix $\mathbf{\Pi}$ to represent the index of the maximum ratio of the available bandwidth of links in the network. Let $\varpi_{k,w}$ be an element in $\mathbf{\Pi}$. For $\forall(k,w) \in \mathcal{L}$, the link (k,w) is congested if the currently occupied bandwidth exceeds $B_{k,w}\varpi_{k,w}$ and less otherwise. Therefore, the number of link congestion at tth time slot is given by

$$f(t) = \sum_{i\in\mathcal{H}} \sum_{x,x'\in\mathcal{F}} \sum_{(k,w)\in\mathcal{L}} n, \ \forall t \in T, \tag{1}$$

where $n = 1$ if $\frac{\sum_{i\in\mathcal{H}} \beta^i_{x,x'} \sigma^{t,k,w}_{i,x,x'}}{B_{k,w}\varpi_{k,w}} > 1$ and 0 otherwise.

C. End-to-End Delay

The end-to-end delay consists of transmission delay, queuing delay, processing delay, and propagation delay. In the network, we assume that the service delay of d_i to traverse the core network is denoted by t^{c}_i (*i.e.,*the delay outside the edge network). At the edge network, the transmission delay here is usually small and is therefore not considered [7]. So the service delay we consider at the edge network includes queuing delay, processing delay, and propagation delay. Here we call the queuing delay and processing delay as "service time".

Service Time: Based on $M/M/1/c$ queuing model [8], we assume that the packet arrives at VNF x is a Poisson Process with mean arrival rate λ_x and the exponential service rate u_x. To ensure the stability of the processing system, the service intensity ρ_x should be less than 1, where $\rho_x = \lambda_x/u_x$. Let $p_{x',x}$ and ϑ^0_x represent the probability of flows from VNF x' to VNF x and the traffic rate of flows from import network elements to VNF x respectively. Then the input traffic rate of VNF x is given by

$$\vartheta_x = \vartheta^0_x + \sum_{x\in\mathcal{F}} \sum_{x'\in\mathcal{F}} \vartheta'_{x'} p_{x',x}. \tag{2}$$

The average number of packets which are being processed by VNF x is given by

$$M_x = \rho_x / (1-\rho_x) = \lambda_x / (u_x - \lambda_x). \tag{3}$$

Therefore, we can obtain the average service time at VNF x in line with Little's Law [10] as follows,

$$T_x = \lambda_x / [\vartheta_x (u_x - \lambda_x)]. \tag{4}$$

Let $Z^i_x = 1$ indicate that S_i requires VNF x and $Z^i_x = 0$ otherwise. Therefore, the service time of d_i is given by

$$t^{\mathrm{s}}_i = \sum_{x\in\mathcal{F}} Z^i_x \times T_x, \forall i \in \mathcal{H}. \tag{5}$$

Propagation delay: We assume that p is the propagation speed of the signal in the physical medium, and $L_{k,w}$ is the physical link length. Then the propagation delay of d_i is given by

$$t^{\mathrm{pro}}_i = \frac{1}{p} \sum_{x\in\mathcal{F}} \sum_{x'\in\mathcal{F}} \sum_{(k,w)\in\mathcal{E}} L_{k,w} \times \sigma^{t,k,w}_{i,x,x'}, \ \forall i \in \mathcal{H}. \tag{6}$$

Therefore, the end-to-end delay of d_i in the network is given by

$$t_i = t^{\mathrm{c}}_i + t^{\mathrm{s}}_i + t^{\mathrm{pro}}_i, \ \forall i \in \mathcal{H}. \tag{7}$$

D. Migration Cost

An NF migration definitely consumes certain bandwidth resources and computing resources. Also, it can increase latency and migration costs [2]. We consider computing resources, bandwidth resources, and latency to be within their respective limits. We assume the maximum migration overhead is donated by C^i_{total}. To avoid frequent service migration, we incorporate migration costs into our model. Let $q^t_{i,k,k'} = 1$ indicate that the user i chooses the node k at time slot $t-1$ and then moves to the node k' at time slot t, and 0 otherwise. Then we consider the following tth time slot migration costs function as [2]:

$$C_t = \sum_{k\in\mathcal{V}} \sum_{k'\in\mathcal{V}} c^t_{i,k,k'} q^t_{i,k,k'}, \tag{8}$$

where the $c^t_{i,k,k'}$ represents the costs of d_i from the previous service node k to the current service node k' at time slot t. So the total migration costs of d_i is given by

$$C^i = \sum_{t\in T} \sum_{k\in\mathcal{V}} \sum_{k'\in\mathcal{V}} c^t_{i,k,k'} q^t_{i,k,k'}. \tag{9}$$

III. Problem Formulation

We divide time T into N time slots in average. Given a set of requests within time T, we investigate the VNF migration problem under network resources constraints, which can be formulated as Problem (10). In Problem (10), $\varepsilon^{g,u,k}_{x,t}$ and $\sigma^{t,k,w}_{i,x,x'}$ are decision variables. Constraint (10.1) states the computing resources limitation, where $W^{t,k}_{g,u}$ and $C'^{t,k}_{g,u}$ represent the occupied computation resource of $V^k_{g,u}$ and the released computation resource of $V^k_{g,u}$ at time t respectively. (10.2)-(10.5) represent the limitation of the link bandwidth, delay, the number of link congestion and migration costs respectively. Moreover, (10.6) and (10.7) are the binary constraints on the decision variables. Constraint (10.8) states the relationship between two variables.

$$\min \sum_{t=0}^{N} f(t) \tag{10}$$

$$\text{s.t.} \sum_{i\in\mathcal{H}} \sum_{x\in\mathcal{F}} \omega^i_x \varepsilon^{g,u,k}_{x,t} + W^{t,k}_{g,u} - C'^{t,k}_{g,u} \le C^k_{g,u}, \forall t \in [0,T], \tag{10.1}$$

$$\sum_{i\in\mathcal{H}} \beta^i_{x,x'} \sigma^{t,k,w}_{i,x,x'} \le B_{k,w}, \forall t \in [0,T], \tag{10.2}$$

$$t_i \le t^{\max}_i, \forall i \in \mathcal{H}, \tag{10.3}$$

$$f(t) \le N_{\max}, \tag{10.4}$$

$$C^i \le C^i_{\text{total}}, \forall i \in \mathcal{H}, \tag{10.5}$$

$$\varepsilon^{g,u,k}_{x,t} \in \{0,1\}, \forall (k,x,t) \in \mathcal{V}\times\mathcal{F}\times T, g \in \{1,2\}, 0 \le u \le y, \tag{10.6}$$

$$\sigma^{t,k,w}_{i,x,x'} \in \{0,1\}, \forall \left(k,w,x,x',i,t\right) \in \mathcal{V}\times\mathcal{V}\times\mathcal{F}\times\mathcal{F}\times\mathcal{H}\times T, \tag{10.7}$$

$$\mathcal{E}^{g,u,k}_{x,t} \mathcal{E}^{g,u,w}_{x,t} = \sigma^{t,k,w}_{i,x,x^1}, g \in \{1,2\}, 0 \le u \le y, \quad \forall \left(k,w,x,x',i,t\right) \in \mathcal{V}\times\mathcal{V}\times\mathcal{F}\times\mathcal{F}\times\mathcal{H}\times T. \tag{10.8}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.

Lemma 1: Problem (10) with constraints (10.1)-(10.8) is NP-hard.

Proof: We consider a special case within a fixed time slot t, which turns the deployment problem into the Multiple-choice Multidimensional Knapsack Problem (MMKP) [9]. Similar to [9], we consider the computing resource of substrate nodes, the bandwidth of substrate links, end-to-end delay, the number of link congestion and migration costs as size vectors, as shown in constraints (10.1) - (10.5). So this special case is NP-hard. Moreover, problem (10) is a dynamic MMKP (DMMKP) problem with t varying, so problem (10) is NP-hard. ■

IV. Solutions

As shown that Problem (10) is NP-hard, there is no polynomial-time algorithm [10]. Hence, in large-scale network scenarios, the computational complexity of finding an optimal solution can be extremely high. Therefore, considering the dynamics of the network, we propose an efficient migration strategy based on DDQN algorithm, called DQNM algorithm, which is shown as Algorithm 1. Compared with the conventional migration strategy, we make differs in the following aspects in this paper:

(1) We combine the MEC clusters and core clouds to improve the overall efficiency of the network.
(2) For the migration strategy in this paper, we simplify the migration schemes by filtering out the redundant migration. For example, the filtered migration of VNFs in S_1 (*i.e.*, $f_{11}^1 \rightarrow f_{12}^1 \rightarrow f_{13}^1 \rightarrow f_{14}^1$) is $\{\{f_{11}^1\}, \{f_{11}^1, f_{12}^1\}, \{f_{11}^1, f_{12}^1, f_{13}^1\}, \{f_{11}^1, f_{12}^1, f_{13}^1, f_{14}^1\}\}$.

We first turn the VNF migration problem into the Markov Decision Process (MDP). An MDP consists of four-tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R)$, where $\mathcal{S}$, $\mathcal{A}$, P, and R represent the set of state space, the set of action space, the transition probability between states, and the reward function, which are defined as follows.

State: The current VNF deployment strategy is used to describe the system states. We assume $s^u \in \mathcal{S}$ represents the network state of the uth MEC cluster, where $\mathcal{S} = \{s^1, s^2, ..., s^u\}$ is the set of all network states. Let $s^u = (s_1^u, s_2^u, s_3^u, ..., s_{N_f}^u)$ be the state space of uth cluster area. For example, $s_x^u = V_{g,u}^k$ represents that VNF x in the uth cluster area is deployed at node $V_{g,u}^k$. N_f is the number of instances of VNF in the uth cluster area. We assume that s_t represents the network state at time slot t.

Action: We first eliminate infeasible actions which exceed the limitation of computing resources. Moreover, we simplify the migration actions to speed up the training by filtering out the redundant actions. Let $\mathcal{A}$ be the set of all actions, where $\mathcal{A} = \{a^1, a^2, ..., a^i\}$. For a specific request d_i, let $a^i = (a_1^i, ..., a_x^i, ..., a_{n_f}^i)$ be the action space to identify the migration action of VNFs of S_i, where x represents the specific VNF of S_i and $a_x^i = V_{g,u}^k$ represents that VNF x of S_i is migrated to the node $V_{g,u}^k$. We assume the first and the last elements of a^i are the ingress and egress network elements of d_i respectively. And a_t represents the action at time slot t.

Transition probability: Let the transition probability be $P = \left\{p_{s^u s^o}^{a^i} | a^i \in \mathcal{A}, s^u, s^o \in \mathcal{S}\right\}$, which represents the probability that network state at the uth cluster area changes from s^u to s^o through action a^i.

Reward: In this paper, combining our optimization goal, we define the opposite of the number of network congestion in time T as the reward (i.e., $r = -\sum_{t=0}^{N} f(t)$).

By introducing immediate rewards of state-action pairs in the MDP, RL adopts the action policy to maximize the long-term rewards. If the system in the state s_t and takes action a_t at time slot t, the state-action value is given by

$$Q(\mathrm{s}_t, \mathrm{a}_t) = \mathbb{E}[\sum_{k=t}^{T} \gamma^k r(\mathrm{s}_t, \mathrm{a}_t) | \mathrm{s}_t, \mathrm{a}_t], \tag{11}$$

where $\gamma \in [0, 1]$ is the discount factor representing the discounted impact of the future reward. A common-used offline strategy algorithm adopts a greedy strategy: $\pi(\mathrm{s}_t) = \mathrm{argmax}_{\mathrm{a}_t} Q(\mathrm{s}_t, \mathrm{a}_t)$.

In order to overcome the drawback with a large state-action space, we employ DQN to approximate the state-action value. The DQN can be trained by minimizing the loss, which is given by

$$L(\theta) = \mathbb{E}[(y_t - Q(\mathrm{s}_t, \mathrm{a}_t | \theta))^2], \tag{12}$$

where θ is the weight vector of DQN and y_t is the target value of the assessment, which is given by

$$y_t = r(\mathrm{s}_t, \mathrm{a}_t) + \gamma Q(\mathrm{s}_t, \mathrm{a}_t | \theta). \tag{13}$$

DDQN adopts the target network parameters in DQN to further reduce the correlation between data. In the process of executing the DDQN algorithm, the network parameters for approximating the action-value function are updated every step, and the parameters for calculating the target value and the target network are updated every C steps. Therefore, the parameter update strategy we use is given by

$$\begin{aligned}\theta_{t+1} = \theta_t + \alpha \Big[& r(\mathrm{s}_t, \mathrm{a}_t) + \gamma Q(\mathrm{s}_{t+1}, \mathrm{argmax}_{\mathrm{a}_t} Q(\mathrm{s}_t, \mathrm{a}_t | \theta_t); \hat{\theta}_t) - \\ & Q(s_t, a_t; \theta_t) \Big] \nabla Q(s_t, a_t; \theta_t),\end{aligned} \tag{14}$$

where $\hat{\theta}$ represents the weight vector of the target network, and the target value in DDQN is given by

$$y_t = r(\mathrm{s}_t, \mathrm{a}_t) + \gamma Q(\mathrm{s}_{t+1}, \mathrm{argmax}_{\mathrm{a}_t} Q(\mathrm{s}_t, \mathrm{a}_t | \theta_t); \hat{\theta}_t). \tag{15}$$

V. Numerical Results

In this section, we use simulation experiments to demonstrate the performance of our online migration strategy. We first assume that users are moving randomly. At the beginning of each slot, the migration decision is made and we assume that the migration time can be negligible. Two existing schemes are used for comparison: 1) O2SP: choose the least number of link congestion in available options at the local cluster area. When the number of requests is relatively small,

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.

this method may get the optimal solution. But with the requests increasing, the complexity of this method is extremely high, and the available options in this method are likely to fall into the local optimum solution. 2) GHA at the edge network: each request is processed individually without constraints violations, for minimizing the number of link congestion caused by each request in the network. In the experiments, the transition probability $p_{x',x}$ is set to 1.

Algorithm 1 DQNM Algorithm

Input: the network state $\mathcal{S}$, the action space $\mathcal{A}$, the reward r, learning rate α, discount factor γ, the target network update interval step C.

output: migration strategy $\mathcal{A}$ and the network state $\mathcal{S}$.

Initialize experience relay pool $\mathcal{D}$, episode M;
Initialize action-value function $Q(\cdot)$ with random weight θ;
Let $\hat{\theta} = \theta$, initialize target action-value function $\hat{Q}(\cdot)$ according to the parameter $\hat{\theta}$;
Receive the initial observed state s_1;
for episode $= 1, 2, ..., M$ **do**
    for $t = 1$ to T **do**
        Randomly select a_t with probability ε;
        Otherwise select $a_t = \text{argmax}_a Q(\cdot)$;
        Execute action a_t, obtain r_t and s_{t+1};
        Store (s_t, a_t, r_t, s_{t+1}) into $\mathcal{D}$;
        Randomly select a sample (s_j, a_j, r_j, s_{j+1}) from the experience relay pool $\mathcal{D}$;
        Set $y_j = r_j$ if episode terminates at step $j+1$, otherwise calculate y_t according to (15);
        Perform a gradient descent step on

$$(y_j - Q(s_j, a_j; \theta))^2,$$

        and update the parameter θ;
        Every C steps reset $\hat{Q} = Q$;
    end for
end for
Obtain migration strategy $\mathcal{A}$ and the network state $\mathcal{S}$.

We assume that each S_i has 4 VNFs. Thus we consider a two-layer fully-connected feedforward neural network, which includes 8 and 25 neurons in the first and second layer respectively. In addition, the activation function is set to the sigmoid function. We assume the probability $\varepsilon = 0.1$, the target network update interval step C is set to 5, the discount factor γ is set to 0.99, and the learning rate α is set to 0.001.

We use the topology of Congent to create the network [10]. We assume there are 100 substrate nodes and 10 clusters. The relative value of computation of MEC service nodes is selected from 10 to 100 randomly. The bandwidth in each cluster is randomly set in $1 \sim 5$ Gbps and the bandwidth between clusters is randomly set in $200 \sim 1000$ Mbps. Moreover, the bandwidth between MEC and core clouds is randomly set in $5 \sim 10$ Gbps. Let the bandwidth threshold of each link be 0.8. The ingress and egress network elements of the same request are randomly selected in the same cluster at the initial time. The propagation delays between substrate nodes at the edge network are set proportional to their geographic distances, perturbed by multiplying a random number in $0.8 \sim 1.2$ [11]. The delay outside the edge network is randomly set in $40 \sim 50$ ms [12]. We generate multiple requests whose minimum traffic rate is 10Mbps [13]. For simplicity, we assume that one hop for migration cost among the MEC is uniformly distributed in [0.1,0.2] cost, and one hop between MEC and core clouds is uniformly distributed in [0.5,1] cost [2]. In this paper, we focus on training one cluster with its corresponding core network area.

First, we explore the relationship between the dynamic changes of link congestion with time-varying. We consider there are 30 requests totally. As shown in Fig. 2, the location of users changes with time varying, and the number of link congestion changes as the location of users. In addition, DQNM in our strategy significantly reduces the number of link congestion, compared with O2SP and GHA. In average, DQNM has an advantage of 16.67% average link congestion over O2SP, and O2SP has an advantage of 42.9% average link congestion over GAH.

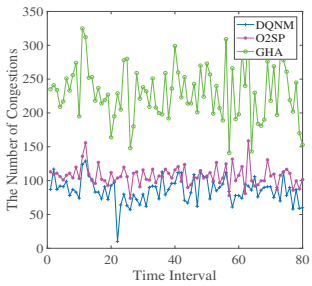

Fig. 2. Congestion with time varying.

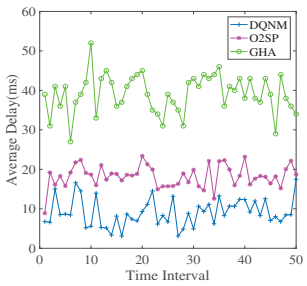

Fig. 3. Average delay varying.

In Fig. 3, we show the average delay of 30 requests. In this experiment, in order to observe the performance of delay, we extend the tolerance delay of the requests. Compared with the GHA, the DQNM and O2SP achieve significant latency reduction. This is because GHA has a relatively high degree of congestion, causing a long processing delay. So even GHA algorithm is executed at the edge network, the average delay is still large. In addition, DQNM outperforms O2SP because O2SP falls into a local optimum solution. Thus O2SP has more congestion in the local cluster.

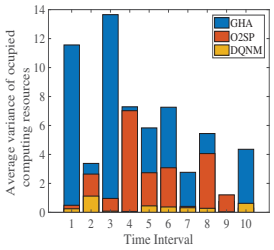

Fig. 4. Nodes balancing.

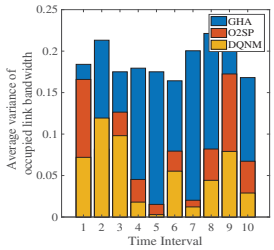

Fig. 5. Links balancing.

Next, we compare the load balancing of the network under different strategies by calculating the average variance of the

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.

occupied computation resources of different service nodes and the average variance of the occupied bandwidth resources of different links. We assume there are 30 service flows in each time slot in the network and the ingress element of each flow is selected randomly. From Fig. 4 and Fig. 5, we can see that DQNM outperforms O2SP and GHA in terms of load balancing. This is because that DQNM can transfer the traffic to the core clouds and adjacent MEC clusters. Moreover, compared with O2SP, DQNM reduces the correlation between the data. Therefore, the number of link congestion of DQNM is fewer than O2SP and GHA.

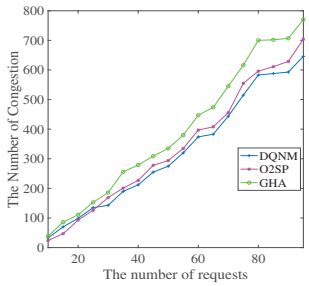

Fig. 6. Congestion with request varying.

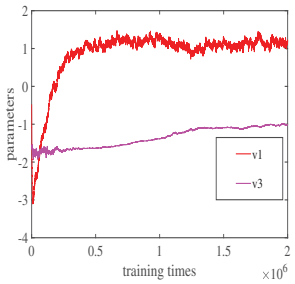

Fig. 7. The connected weights.

Then, we show the relationship between the dynamic variations of link congestion with request increasing. From Fig. 6, we can see that the number of link congestion increases as the number of requests increases. In addition, when the number of requests is small, O2SP chooses the least congestion schemes among the rare possible schemes, which is closer to the optimal solution. However, with the number of requests increasing, compared with O2SP and GHA, DQNM in our strategy significantly reduces the number of link congestion. This is because that the DQNM reduces the correlation between the data with requests increasing. Moreover, DQNM combines core clouds and the adjacent clusters to reduce the congestion in the network.

Finally, we show the convergence of neural network parameters in the DQNM algorithm. We randomly select two connected weights in the neural networks. We demonstrate the relationship between the connected weights and training times in Fig. 7. As shown in Fig. 7, the weight parameters tend to stabilize after approximately 1.3 million training, which indicates the convergence of DQNM algorithm. It is worth noting that although the DQNM algorithm is an online learning algorithm, where the agent can collect training information and train its strategy in the gap between every two decision time points, thus accelerating the convergence of the algorithm. In addition, the training strategy can be applied to similar scenarios. Therefore, the DQNM algorithm can efficiently perform and make migration decisions in accordance with the policies it has learned.

In summary, the simulation results show that DQNM and O2SP always outperform GHA. This is because that DQNM and O2SP not only consider the VNF migration in the network with MEC but also the cooperation with the adjacent clusters. In other words, GHA considers the requests at the edge network only, which causes heavy traffic congestion so that an increased processing delay occurs. Moreover, DQNM always outperforms O2SP. Because compared with O2SP, DQNM reduces the correlation between the data. Also, O2SP is likely to fall into the local optimum solution in large-scale network because of the increasing of requests.

VI. Conclusion

In this paper, we have proposed an online migration strategy in softwarization based mobile networks with MEC to support diverse services with various Quality of Service requirements. We formulate the VNF migration as an optimization problem aiming at minimizing the number of link congestion of all requests in the current system. In order to efficiently solve this problem, we resort to DRL to overcome the large state space and action space of the MDP in our migration strategy. Through simulations, we have validated the effectiveness of our proposed cooperative migration strategy. Numerical results have shown that our proposed solution outperforms existing O2SP and GHA in terms of the number of link congestion, average delay, and load balance.

References

[1] Z. Li, M. A. Unsitalo, H. Shariatmadari, and B. Singh, "5G URLLC: Design Challenges and System Concepts," 2018 15th International Symposium on Wireless Communication Systems (ISWCS), Lisbon, Portugal, 2018, pp. 1-6

[2] T. Ouyang, R. Li, X. Chen, Z. Zhou and X. Tang, "Adaptive User-managed Service Placement for Mobile Edge Computing: An Online Learning Approach," IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, Paris, France, 2019, pp. 1468-1476.

[3] X. Cao, G. Tang, D. Guo, et al. Edge Federation: Towards an Integrated Service Provisioning Model[J]. arXiv preprint arXiv:1902.09055, 2019.

[4] L. Zhang, J. Tan, Y. Liang, G. Feng and D. Niyato, "Deep Reinforcement Learning-Based Modulation and Coding Scheme Selection in Cognitive Heterogeneous Networks," IEEE Transactions on Wireless Communications, vol. 18, no. 6, pp. 3281-3294, June 2019.

[5] R. Gouareb, V. Friderikos and A. Aghvami, "Virtual Network Functions Routing and Placement for Edge Cloud Latency Minimization," IEEE Journal on Selected Areas in Communications, vol. 36, no. 10, pp. 2346-2357, Oct. 2018.

[6] F. Z. Yousaf, M. Bredel, S. Schaller and F. Schneider, "NFV and SDN—Key Technology Enablers for 5G Networks," IEEE Journal on Selected Areas in Communications, vol. 35, no. 11, pp. 2468-2478, Nov. 2017.

[7] R. Ramaswamy, N. Weng, T. Wolf. "Characterizing network processing delay." IEEE Global Telecommunications Conference, 2004. GLOBECOM'04. Vol. 3. IEEE, 2004.

[8] A. Y. Khinchin, D. M. Andrews, M. H. Quenouille. Mathematical methods in the theory of queuing[M]. Courier Corporation, 2013.

[9] A. Sbihi. A best-first search exact algorithm for the multiple-choice multidimensional knapsack problem[J]. Journal of Combinatorial Optimization, 2007, 13(4): 337-351.

[10] The Cogent's Network Map. Accessed: 2009. [Online]. Available: http://cogentco.com/en/network/network-map.

[11] Y. Jia, C. Wu, Z. Li, F. Le and A. Liu, "Online Scaling of NFV Service Chains Across Geo-Distributed Datacenters," IEEE/ACM Transactions on Networking, vol. 26, no. 2, pp. 699-710, April 2018.

[12] I. Parvez, A. Rahmati, I. Guvenc, A.I. Sarwat and H.Dai, "A Survey on Low Latency Towards 5G: RAn, Core Network and Caching Solution," IEEE Communication Surveys Tutorials, 2018, 20(4):3098-3130.

[13] X. Li and C. Qian, "Low-complexity multi-resource packet scheduling for network function virtualization," IEEE INFOCOM, 2015.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 03:35:44 UTC from IEEE Xplore. Restrictions apply.